



Des interfaces aux Streams API

Programmation orientée objet et programmation fonctionnelle en Java

RABOANARY Heriniaina Andry

Public : L2 Informatique

Prérequis : classes, objets, héritage, collections ArrayList

Version de Java : Java 8 ou ultérieure

Année universitaire : 2025-2026

Objectifs du chapitre

À la fin de ce chapitre, l'étudiant doit être capable de :

- comprendre le rôle des interfaces en Java ;
- distinguer interface classique et interface fonctionnelle ;
- utiliser les expressions lambda ;
- utiliser les références de méthodes ;
- comprendre le principe des Streams ;
- appliquer les principales opérations des Streams API ;
- manipuler des collections de manière plus moderne et plus lisible.

Introduction

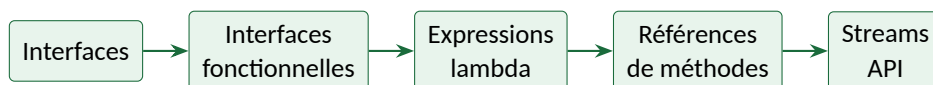
Java est un langage principalement orienté objet. Les notions de classe, objet, héritage, encapsulation et polymorphisme sont donc fondamentales.

Cependant, depuis Java 8, le langage a introduit des mécanismes inspirés de la programmation fonctionnelle :

- les interfaces fonctionnelles ;
- les expressions lambda ;
- les références de méthodes ;
- les Streams API.

Ces outils permettent d'écrire un code plus court, plus lisible et plus expressif, surtout lorsqu'on manipule des collections d'objets.

La progression logique de ce chapitre part du contrat le plus général — l'interface — pour aboutir au traitement déclaratif des collections :



1. Les interfaces

1.1 Définition

Une interface est un **contrat**. Elle indique *ce qu'une classe doit savoir faire*, sans imposer *comment* elle doit le faire.

```
public interface Animal {  
    void crier();  
}
```

Cette interface impose à toute classe qui l'implémente de définir la méthode `crier()`.

1.2 Implémentation d'une interface

```
public class Chien implements Animal {
    @Override
    public void crier() {
        System.out.println("Wouf");
    }
}

public class Chat implements Animal {
    @Override
    public void crier() {
        System.out.println("Miaou");
    }
}
```

Les classes Chien et Chat respectent le contrat défini par l'interface Animal.

1.3 Interface et polymorphisme

Grâce aux interfaces, on peut écrire :

```
Animal a1 = new Chien();
Animal a2 = new Chat();
a1.crier(); // affiche : Wouf
a2.crier(); // affiche : Miaou
```

La variable est de type Animal, mais l'objet réel peut être un Chien, un Chat ou toute autre classe implémentant Animal. C'est une forme importante du polymorphisme.

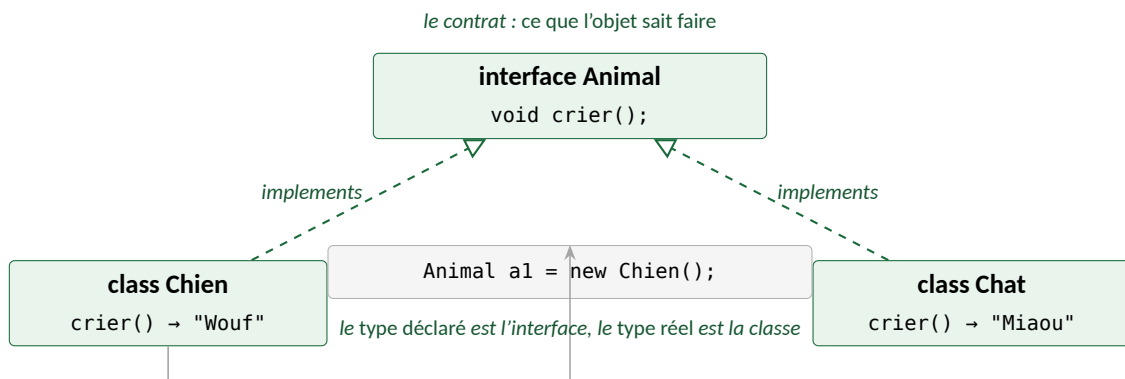


Fig. 1 : Une interface sépare le contrat de sa réalisation. Le code client ne connaît que Animal.

1.4 Méthode recevant une interface

Une méthode peut recevoir une interface en paramètre :

```
public void faireCrier(Animal animal) {
    animal.crier();
}
```

On peut ensuite écrire :

```
faireCrier(new Chien());
faireCrier(new Chat());
```

La méthode ne dépend pas d'une classe précise. Elle dépend seulement du contrat `Animal`. C'est ce découplage qui permettra, plus loin, de passer une simple expression lambda à la place d'un objet.

1.5 Interface ou classe abstraite ?

Critère	Interface	Classe abstraite
Rôle principal	Définit surtout un contrat	Définit une base commune
Héritage	Une classe peut implémenter plusieurs interfaces	Une classe ne peut hériter que d'une seule classe abstraite
Objectif	Favorise le polymorphisme	Favorise la factorisation de code
Usage	Très utilisée dans les architectures modernes	Utile lorsqu'il existe du code commun

En général :

- on utilise une **interface** pour dire *ce qu'un objet sait faire* ;
- on utilise une **classe abstraite** lorsqu'on veut *partager du code* entre plusieurs classes proches.

2. Les interfaces fonctionnelles

2.1 Définition

Une interface fonctionnelle est une interface qui possède **une seule méthode abstraite**.

```
@FunctionalInterface
public interface Calcul {
    int calculer(int a, int b);
}
```

Cette interface possède une seule méthode abstraite : `calculer`. Elle est donc compatible avec les expressions lambda.

2.2 Rôle de `@FunctionalInterface`

L'annotation `@FunctionalInterface` n'est pas obligatoire, mais elle est recommandée. Elle demande au compilateur de vérifier que l'interface contient bien une seule méthode abstraite.

Exemple incorrect :

```
@FunctionalInterface
public interface Calcul {
    int additionner(int a, int b);
    int multiplier(int a, int b);
}
```

Ce code provoque une erreur, car l'interface possède deux méthodes abstraites. Le compilateur (javadoc, Java 21) répond exactement :

```
error: Unexpected @FunctionalInterface annotation
    Calcul is not a functional interface
```

```
multiple non-overriding abstract methods found in interface Calcul
```

2.3 Pourquoi une seule méthode ?

Une expression lambda représente une implémentation courte d'une méthode. Java doit donc savoir à quelle méthode correspond la lambda. C'est pourquoi une interface fonctionnelle ne doit avoir qu'une seule méthode abstraite : sans ambiguïté possible, le compilateur peut relier la lambda à sa signature.

3. Les expressions lambda

3.1 Avant les lambdas

Avant Java 8, pour implémenter rapidement une interface, on utilisait souvent une classe anonyme.

```
Calcul addition = new Calcul() {  
    @Override  
    public int calculer(int a, int b) {  
        return a + b;  
    }  
};
```

Ce code est correct, mais il est long : six lignes pour exprimer une addition.

3.2 Avec une lambda

Depuis Java 8, on peut écrire :

```
Calcul addition = (a, b) → a + b;
```

La lambda signifie : *prendre a et b, puis retourner a + b.*

3.3 Syntaxe générale

```
(parametres) → expression
```

ou bien :

```
(parametres) → {  
    instructions;  
    return valeur;  
}
```

Exemples :

```
x → x * x  
(a, b) → a + b  
(a, b) → {  
    int resultat = a + b;  
    return resultat;  
}
```

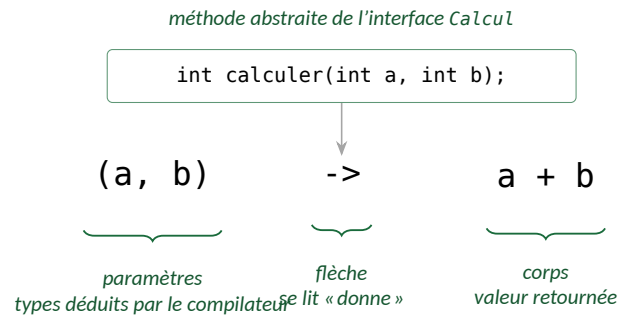


Fig. 2 : Anatomie d'une expression lambda. La lambda ne porte pas de nom : c'est la signature de l'unique méthode abstraite de l'interface cible qui lui donne son sens.

3.4 Exemples d'utilisation

```
Calcul addition      = (a, b) → a + b;
Calcul multiplication = (a, b) → a * b;
Calcul soustraction  = (a, b) → a - b;

System.out.println(addition.calculer(4, 2));    // 6
System.out.println(multiplication.calculer(4, 2)); // 8
System.out.println(soustraction.calculer(4, 2)); // 2
```

3.5 Lambda sans paramètre

```
Runnable action = () → System.out.println("Bonjour Java");
```

Ici, la lambda ne reçoit aucun paramètre : les parenthèses vides sont obligatoires.

3.6 Lambda avec un seul paramètre

```
Consumer<String> afficher = message → System.out.println(message);
```

Quand il n'y a qu'un seul paramètre, les parenthèses sont facultatives.

3.7 Lambda avec plusieurs paramètres

```
BinaryOperator<Integer> somme = (a, b) → a + b;
```

Quand il y a plusieurs paramètres, les parenthèses sont obligatoires.

4. Les interfaces fonctionnelles de `java.util.function`

Java fournit déjà plusieurs interfaces fonctionnelles prêtes à l'emploi dans le package `java.util.function`. Il est donc rarement nécessaire d'écrire soi-même une interface fonctionnelle.

Interface	Méthode	Rôle
Consumer<T>	accept(T t)	Consomme une valeur (aucun retour)
Supplier<T>	get()	Produit une valeur
Predicate<T>	test(T t)	Teste une condition (retourne un booléen)
Function<T, R>	apply(T t)	Transforme une valeur de type T en type R
UnaryOperator<T>	apply(T t)	Transforme une valeur en conservant le même type
BinaryOperator<T>	apply(T a, T b)	Combine deux valeurs du même type

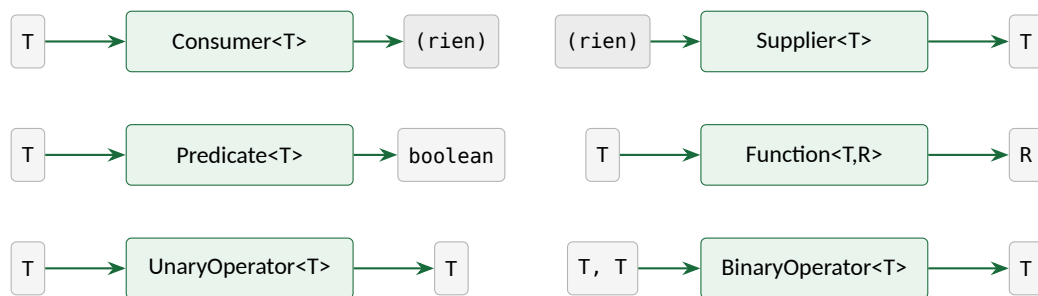


Fig. 3 : Les six interfaces fonctionnelles usuelles se distinguent uniquement par ce qu'elles reçoivent et ce qu'elles rendent.

4.1 Consumer

Un Consumer reçoit une valeur et ne retourne rien.

```
Consumer<String> afficher = s → System.out.println(s);
afficher.accept("Bonjour"); // Bonjour
```

4.2 Supplier

Un Supplier ne reçoit rien et retourne une valeur.

```
Supplier<Double> aleatoire = () → Math.random();
System.out.println(aleatoire.get());
```

4.3 Predicate

Un Predicate reçoit une valeur et retourne un booléen.

```
Predicate<Integer> estPair = x → x % 2 == 0;
System.out.println(estPair.test(8)); // true
```

4.4 Function

Une Function transforme une valeur en une autre valeur.

```
Function<String, Integer> longueur = s → s.length();
System.out.println(longueur.apply("Java")); // 4
```

4.5 UnaryOperator

Un `UnaryOperator` transforme une valeur en une autre valeur du même type.

```
UnaryOperator<Integer> carre = x → x * x;
System.out.println(carre.apply(5)); // 25
```

4.6 BinaryOperator

Un `BinaryOperator` combine deux valeurs du même type.

```
BinaryOperator<Integer> somme = (a, b) → a + b;
System.out.println(somme.apply(3, 7)); // 10
```

5. Les références de méthodes

5.1 Définition

Une référence de méthode est une écriture simplifiée d'une lambda qui appelle directement une méthode existante.

Au lieu d'écrire `s -> System.out.println(s)`, on peut écrire `System.out::println`.

5.2 Exemple avec `forEach`

```
ArrayList<String> noms = new ArrayList<>();
noms.add("Alice");
noms.add("Bob");
noms.add("Charles");

noms.forEach(System.out::println);
// Alice
// Bob
// Charles
```

5.3 Principales formes

Forme	Exemple	Lambda équivalente
Méthode statique	<code>Math::abs</code>	<code>x -> Math.abs(x)</code>
Méthode d'un objet précis	<code>System.out::println</code>	<code>s -> System.out.println(s)</code>
Méthode d'un type	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>
Constructeur	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>

5.4 Exemple avec `String::toUpperCase`

```
ArrayList<String> noms = new ArrayList<>();
noms.add("alice");
noms.add("bob");

noms.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

```
// ALICE  
// BOB
```

À retenir

Une référence de méthode ne s'utilise que si la lambda *se contente* d'appeler une méthode existante avec les mêmes arguments. Dès qu'il y a un calcul supplémentaire, il faut revenir à la lambda : `s -> s.toUpperCase() + "!"` ne peut pas s'écrire en référence de méthode.

6. Les Streams API

6.1 Définition

Un `Stream` est une séquence d'éléments sur laquelle on peut appliquer des traitements.

Remarque importante

Un `Stream` **n'est pas une collection**. Il ne stocke pas les données : il permet seulement de les parcourir et de les traiter. Une `ArrayList` occupe de la mémoire ; un `Stream` décrit un traitement.

6.2 Pourquoi utiliser les Streams ?

Les Streams permettent d'écrire un code plus *déclaratif*. Avec une boucle classique, on explique *comment* faire (style impératif). Avec un `Stream`, on explique *ce qu'on veut obtenir* (style déclaratif).

Exemple avec boucle classique :

```
for (String nom : noms) {  
    if (nom.startsWith("A")) {  
        System.out.println(nom);  
    }  
}
```

Exemple avec `Stream` :

```
noms.stream()  
    .filter(nom -> nom.startsWith("A"))  
    .forEach(System.out::println);
```

6.3 Structure d'un pipeline `Stream`

Un pipeline `Stream` contient généralement trois parties : une **source**, une ou plusieurs **opérations intermédiaires**, et une **opération terminale**.

```
noms.stream()  
    .filter(n -> n.startsWith("A"))  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Dans cet exemple :

- `noms.stream()` est la source;

- filter et map sont des opérations intermédiaires;
- forEach est l'opération terminale.

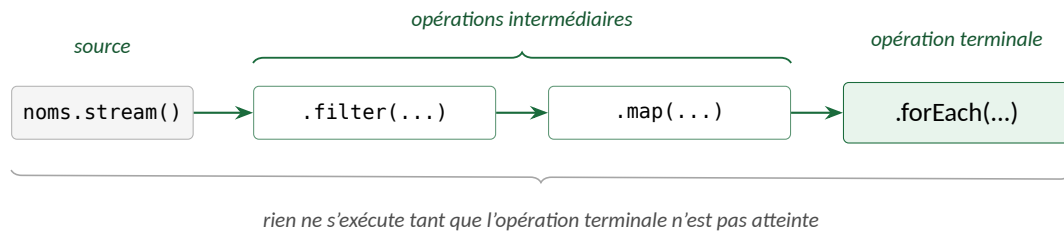


Fig. 4 : Anatomie d'un pipeline Stream. Chaque opération intermédiaire rend un *nouveau* Stream et se contente de décrire le traitement; seule l'opération terminale le déclenche et produit un résultat.

6.4 Un Stream ne modifie pas la collection d'origine

```
ArrayList<String> noms = new ArrayList<>();
noms.add("alice");
noms.add("bob");

noms.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println); // ALICE puis BOB

System.out.println(noms);         // [alice, bob]
```

La liste noms contient toujours "alice" et "bob". Le Stream produit un traitement, mais ne modifie pas la collection originale.

6.5 Un Stream ne peut pas être réutilisé

```
Stream<String> s = noms.stream();
s.forEach(System.out::println);
s.count(); // Erreur !
```

Après une opération terminale, le Stream est consommé. L'exécution s'interrompt avec :

```
java.lang.IllegalStateException:
stream has already been operated upon or closed
```

Il faut créer un nouveau Stream si nécessaire.

6.6 L'évaluation est paresseuse

Les éléments ne traversent pas le pipeline « par étages » : chaque élément le parcourt *entièrement* avant que le suivant ne commence. On l'observe avec peek :

```
noms.stream()
    .peek(n → System.out.println(" peek → " + n))
    .map(String::toUpperCase)
    .forEach(n → System.out.println(" forEach → " + n));
```

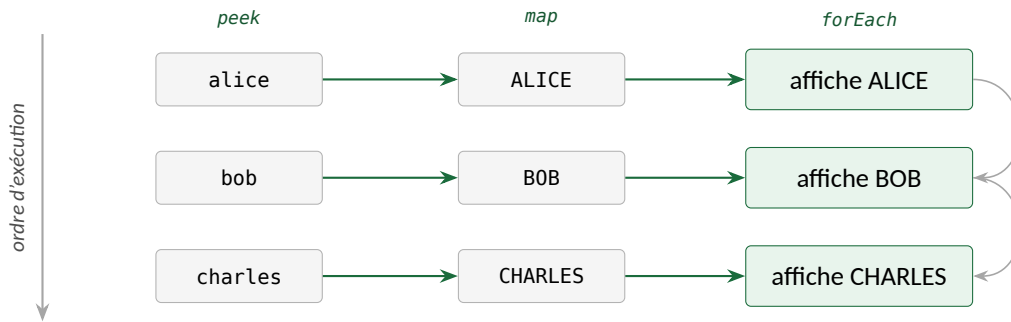


Fig. 5 : Évaluation paresseuse : alice traverse tout le pipeline avant que bob n'y entre. Sortie réelle vérifiée avec Java 21.

7. Les opérations intermédiaires

Les opérations intermédiaires transforment un Stream en un autre Stream. Elles ne produisent pas directement de résultat final.

filter conserve seulement les éléments qui vérifient une condition.

map transforme chaque élément.

sorted trie les éléments.

distinct supprime les doublons.

limit conserve seulement les n premiers éléments.

skip ignore les n premiers éléments.

peek permet d'observer les éléments pendant le traitement (surtout pour le débogage).

flatMap aplatit plusieurs collections en un seul Stream.

Toutes les sorties ci-dessous ont été obtenues avec la liste :

```
ArrayList<Integer> notes =
    new ArrayList<>(List.of(14, 8, 12, 16, 9, 12, 18, 7));
```

```
notes.stream().filter(note → note >= 10)... // 14 12 16 12 18
notes.stream().map(note → note + 1)...      // 15 9 13 17 10 13 19 8
notes.stream().sorted()...                  // 7 8 9 12 12 14 16 18
notes.stream().sorted((a, b) → b - a)...    // 18 16 14 12 12 9 8 7
notes.stream().distinct()...                // 14 8 12 16 9 18 7
notes.stream().limit(3)...                  // 14 8 12
notes.stream().skip(2)...                   // 12 16 9 12 18 7
```

flatMap sert lorsqu'un élément contient lui-même une collection :

```
ArrayList<ArrayList<String>> groupes = new ArrayList<>();
// [[Alice, Bob], [Charles], [Dina, Eva]]

groupes.stream()
    .flatMap(groupe → groupe.stream())
    .forEach(System.out::println);
// Alice / Bob / Charles / Dina / Eva
```

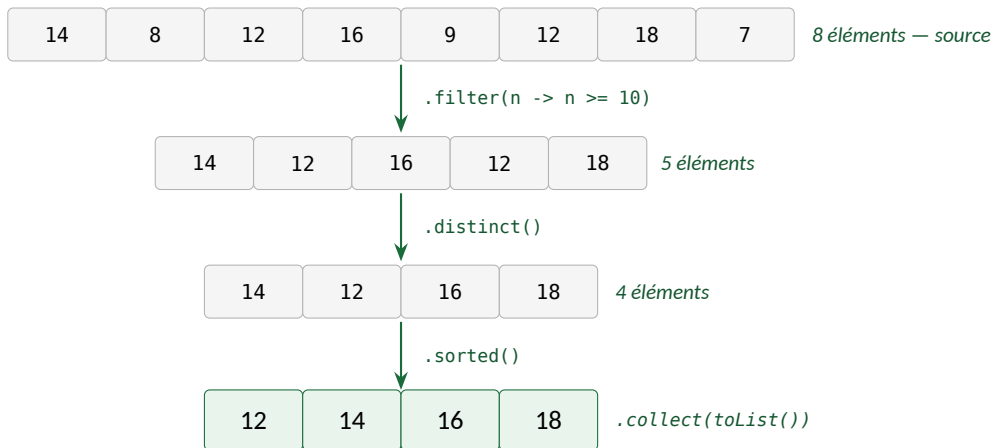


Fig. 6 : Trois opérations intermédiaires enchaînées sur la liste des notes. Valeurs obtenues par exécution effective du code.

8. Les opérations terminales

Les opérations terminales produisent un résultat final. Après une opération terminale, le Stream est consommé.

forEach — affiche ou traite chaque élément.

```
noms.stream().forEach(System.out::println);
```

count — compte les éléments.

```
long nombre = noms.stream().count();           // 3
long admis  = notes.stream().filter(n -> n >= 10).count(); // 5
```

min / max — retournent le plus petit ou le plus grand élément, sous forme d'Optional.

```
Optional<Integer> min = notes.stream().min(Integer::compareTo);
Optional<Integer> max = notes.stream().max(Integer::compareTo);
System.out.println(min);    // Optional[7]
System.out.println(max);    // Optional[18]
```

reduce — combine tous les éléments pour obtenir une seule valeur.

```
int somme          = notes.stream().reduce(0, (a, b) -> a + b); // 96
int sommeAlternative = notes.stream().reduce(0, Integer::sum);   // 96
```

collect — reconstruit une collection ou produit un résultat structuré.

```
List<String> nomsA = noms.stream()
    .filter(n -> n.startsWith("A"))
    .collect(Collectors.toList()); // [Alice]
```

Pourquoi un Optional ?

min et max rendent un Optional parce que le Stream peut être vide : il n'y aurait alors aucun maximum à retourner. Appeler get() sur un Optional vide lève NoSuchElementException. Préférer ifPresent(...) ou orElse(...) :

```
Optional<Integer> m = vide.stream().max(Integer::compareTo);
System.out.println(m);           // Optional.empty
System.out.println(m.orElse(-1)); // -1
```

9. Les Collectors (java.util.stream.Collectors)

toList() / **toSet()** récupèrent les résultats sous forme de List ou de Set.

joining() concatène des chaînes de caractères.

```
String texte = noms.stream().collect(Collectors.joining(", "));
// "Alice, Bob, Charles"
```

counting() compte les éléments.

averagingDouble() calcule une moyenne.

```
double moyenne = etudiants.stream()
    .collect(Collectors.averagingDouble(Etudiant::getMoyenne));
```

groupingBy() regroupe les éléments selon un critère.

```
Map<String, List<Etudiant>> parNiveau = etudiants.stream()
    .collect(Collectors.groupingBy(Etudiant::getNiveau));
```

partitioningBy() divise les éléments en deux groupes (true / false) selon une condition.

```
Map<Boolean, List<Etudiant>> resultat = etudiants.stream()
    .collect(Collectors.partitioningBy(e -> e.getMoyenne() >= 10));
```

summarizingDouble() obtient plusieurs statistiques en une seule passe.

```
DoubleSummaryStatistics stats = etudiants.stream()
    .collect(Collectors.summarizingDouble(Etudiant::getMoyenne));
// getMin() getMax() getAverage() getSum() getCount()
```

10. Étude de cas : gestion d'étudiants

10.1 Classe Etudiant

```
1 public class Etudiant {
2     private String nom;
3     private String niveau;
4     private double moyenne;
5
6     public Etudiant(String nom, String niveau, double moyenne) {
7         this.nom = nom;
8         this.niveau = niveau;
9         this.moyenne = moyenne;
```

```

10     }
11
12     public String getNom()      { return nom; }
13     public String getNiveau()  { return niveau; }
14     public double getMoyenne() { return moyenne; }
15
16     @Override
17     public String toString() {
18         return nom + " - " + niveau + " - " + moyenne;
19     }
20 }

```

10.2 Exemples de manipulations

```

ArrayList<Etudiant> etudiants = new ArrayList<>();
etudiants.add(new Etudiant("Alice", "L2", 14.5));
etudiants.add(new Etudiant("Bob", "L2", 8.0));
etudiants.add(new Etudiant("Charles", "L3", 12.0));
etudiants.add(new Etudiant("Dina", "M1", 16.0));
etudiants.add(new Etudiant("Eva", "M1", 9.5));

// Afficher tous les etudiants
etudiants.forEach(System.out::println);

// Afficher les etudiants admis
etudiants.stream()
    .filter(e → e.getMoyenne() >= 10)
    .forEach(System.out::println);
// Alice - L2 - 14.5 / Charles - L3 - 12.0 / Dina - M1 - 16.0

// Trier les etudiants par moyenne croissante
etudiants.stream()
    .sorted((e1, e2) → Double.compare(e1.getMoyenne(), e2.getMoyenne()))
    .forEach(System.out::println);
// Bob / Eva / Charles / Alice / Dina

// Trier les etudiants par moyenne decroissante
etudiants.stream()
    .sorted((e1, e2) → Double.compare(e2.getMoyenne(), e1.getMoyenne()))
    .forEach(System.out::println);
// Dina / Alice / Charles / Eva / Bob

// Calculer la moyenne generale
double moyenneGenerale = etudiants.stream()
    .collect(Collectors.averagingDouble(Etudiant::getMoyenne));
// 12.0

// Trouver le meilleur etudiant
Optional<Etudiant> meilleur = etudiants.stream()
    .max((e1, e2) → Double.compare(e1.getMoyenne(), e2.getMoyenne()));
meilleur.ifPresent(System.out::println);
// Dina - M1 - 16.0

// Regrouper les etudiants par niveau
Map<String, List<Etudiant>> parNiveau = etudiants.stream()
    .collect(Collectors.groupingBy(Etudiant::getNiveau));

// Separer admis et non admis

```

```
Map<Boolean, List<Etudiant>> decision = etudiants.stream()
    .collect(Collectors.partitioningBy(e -> e.getMoyenne() >= 10));

// Obtenir les statistiques des moyennes
DoubleSummaryStatistics stats = etudiants.stream()
    .collect(Collectors.summarizingDouble(Etudiant::getMoyenne));
// count=5, sum=60.0, min=8.0, average=12.0, max=16.0
```

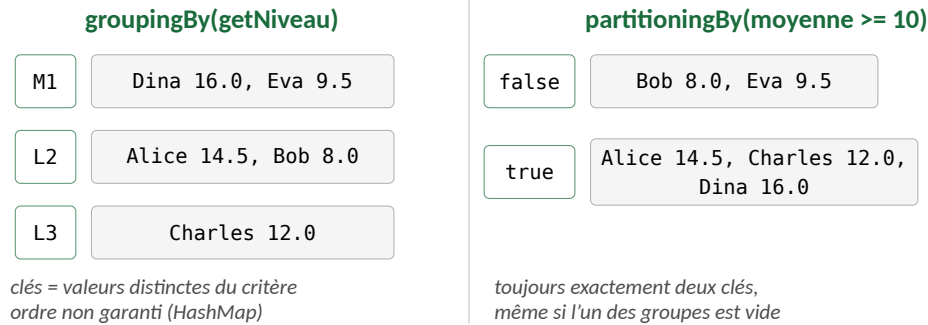


Fig. 7 : `groupingBy` produit autant de groupes que de valeurs du critère; `partitioningBy` en produit toujours deux.

11. Bonnes pratiques

Utiliser les lambdas pour simplifier le code. Préférer les expressions courtes (`n -> n.startsWith("A")`). Si le traitement devient trop long, écrire une méthode séparée et la référencer.

Ne pas abuser des Streams. Les Streams rendent le code plus lisible pour des traitements simples ou moyens. Mais une boucle classique peut parfois être plus claire pour un traitement complexe à étapes multiples.

Bien nommer les variables. Préférer `.filter(note -> note >= 10)` à `.filter(x -> x > 10)`.

Ne pas modifier une collection dans un Stream. Éviter d'ajouter ou de supprimer des éléments dans la collection d'origine pendant le traitement par Stream.

Quatre pièges fréquents en TP

1. **Stream réutilisé** — un Stream stocké dans une variable ne sert qu'une fois; repartir de `collection.stream()`.
2. **Pipeline sans opération terminale** — sans `forEach`, `collect` ou `count`, *rien ne s'exécute* et aucune erreur n'apparaît.
3. **Optional non déballé** — afficher `max` directement donne `Optional[18]` et non `18`.
4. **Ordre de groupingBy** — le résultat est une `HashMap` : ne jamais supposer que les clés sortent dans l'ordre alphabétique.

Les cinq sujets de travaux pratiques de ce chapitre sont regroupés, avec leurs corrigés, dans le fascicule « Travaux pratiques et corrigés » qui accompagne ce polycopié.

Pour aller plus loin

- `Comparator.comparing(Etudiant::getMoyenne)` et `.reversed()` remplacent avantageusement les lambdas de tri écrites à la main.
- `IntStream`, `DoubleStream` évitent l'autoboxing et offrent `sum()`, `average()` directement.
- `Collectors.groupingBy(critere, Collectors.counting())` combine deux collecteurs.
- `parallelStream()` distribue le traitement sur plusieurs cœurs — utile seulement sur de très grands volumes, et à manier avec prudence.

Résumé général**L'essentiel du chapitre**

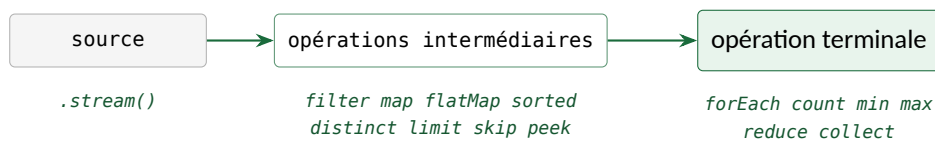
Interface définit un contrat que les classes doivent respecter.

Interface fonctionnelle interface qui possède une seule méthode abstraite.

Expression lambda permet d'implémenter rapidement une interface fonctionnelle.

Référence de méthode écriture abrégée `Classe::methode` d'une lambda qui se contente d'appeler une méthode existante.

Streams API permet de traiter des collections de manière déclarative.



Collectors — permettent de reconstruire, regrouper ou résumer les données : `toList`, `toSet`, `joining`, `counting`, `averagingDouble`, `groupingBy`, `partitioningBy`, `summarizingDouble`.

Terme	À retenir
Contrat	Ce qu'une classe s'engage à savoir faire, sans dire comment.
Méthode abstraite	Méthode déclarée sans corps ; l'unique méthode abstraite d'une interface fonctionnelle sert de cible aux lambdas.
Pipeline	Enchaînement source → intermédiaires → terminale.
Paresse	Aucune opération intermédiaire ne s'exécute avant l'opération terminale.
Consommation	Un Stream ne se parcourt qu'une seule fois.
Optional	Conteneur qui représente « une valeur ou rien » ; se déballe avec <code>ifPresent</code> ou <code>orElse</code> .